

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

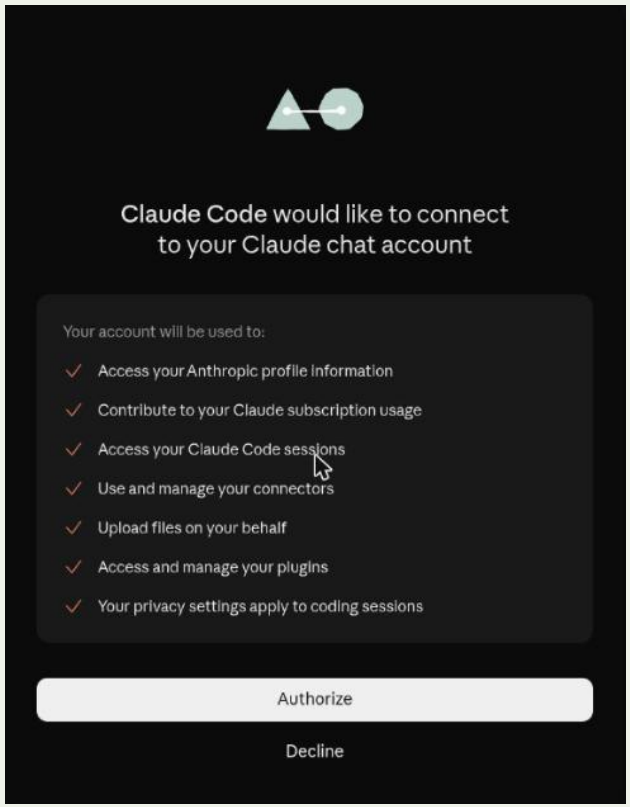
Modulo 4 — Un agente nel repository

Installare e configurare Claude Code

Installare: Linux e WSL 2

Fatto prima del corso; qui lo ripercorriamo

- Prima del corso avete installato Claude Code con la guida `PRIMA_DEL_CORSO.md`, e `tools/verifica_ambiente.sh` l'ha controllato: qui lo **ripercorriamo**, per intero
- L'installazione **nativa**, quella consigliata, anche in WSL: `curl -fsSL https://claude.ai/install.sh | bash`
- In WSL 2 si installa e si avvia **dal terminale WSL**, non da PowerShell o CMD; i progetti stanno nel filesystem **Linux**, sotto `/home/`, non sotto `/mnt/c/`, dove la ricerca trova meno del previsto
- Se la shell non trova `claude`, la cartella `~/.local/bin` non è ancora nel PATH: l'installer stesso stampa il comando che la aggiunge
- Gli aggiornamenti arrivano **in background**, e valgono dal riavvio
- Al primo avvio `claude` apre il **browser**: Claude Code chiede di collegarsi al vostro account, e voi autorizzate. Se il browser mostra un **codice** invece di tornare al terminale, incollatelo nel terminale: in WSL 2 succede spesso



Il primo accesso di Claude Code nel browser; schermata del docente, 28 ottobre 2026

Controllare: tre comandi

La versione, la diagnostica, l'accesso

1. `claude --version`: un'installazione funzionante stampa la sua **versione**; sulla macchina del docente, `2.1.312 (Claude Code)`
2. `claude doctor`: una diagnostica dell'installazione e delle impostazioni, **in sola lettura**, senza aprire una sessione; dentro una sessione, `/doctor` fa un controllo completo, e sa anche correggere. Nel riquadro, la **registrazione del corso**: 28 ottobre 2026, Claude Code 2.1.312

`claude doctor: «[…]» segna un taglio`

```
[…]
Running: native (2.1.312)
[…]
Path: ~/.local/share/claude/versions/2.1.312
Config install method: native
Search: OK (bundled)
Auto-updates: enabled
[…]
No installation issues found.
[…]
```

3. `claude auth status`: lo stato dell'**accesso**, in JSON, oppure in testo leggibile con `--text`; esce con 0 se l'accesso c'è, con 1 se no. Il metodo è nel campo `authMethod`: per esempio `claude.ai`, oppure `api_key`

Il Pro, e la chiave che lo scavalca

Un abbonamento per Claude e per Claude Code; e una variabile che passa davanti

- Con il Pro, **un solo abbonamento**: Claude sul web, sul desktop e sul telefono, e Claude Code. Per chi ha il Pro il metodo predefinito è l'**abbonamento**, con l'accesso fatto nel browser
- Se le credenziali sono più d'una, Claude Code ne sceglie una **in ordine**: la variabile `ANTHROPIC_API_KEY` è al **terzo** posto, l'abbonamento al **settimo**
- Con `ANTHROPIC_API_KEY` impostata, Claude Code usa **la chiave** al posto dell'abbonamento, anche se avete fatto l'accesso; e l'uso si paga **come API**, che il Pro non include. Con `claude -p` la chiave, se c'è, si usa **sempre**; in una sessione interattiva, dopo che l'avete approvata
- Il sintomo: «This organization has been disabled», con un abbonamento attivo
- Il rimedio: `unset ANTHROPIC_API_KEY`, e `/status` per vedere quale metodo è attivo

M4-D01 La chiave che scavalca il Pro

Demo del docente nel terminale, poi la vostra prova: nessun messaggio a Claude

⚖ Due esecuzioni dello stesso comando, `claude auth status --json`, che differiscono in una cosa sola: la **chiave**. La prima senza `ANTHROPIC_API_KEY`; la seconda con una chiave **finta**, impostata per quel solo comando. `controlla_accesso.py --salva` ne tiene soltanto tre campi, mai i dati dell'account

```
env -u ANTHROPIC_API_KEY claude auth status --json \
  | python3 controlla_accesso.py --salva stato/senza.json
env ANTHROPIC_API_KEY=finta-chiave-del-corso \
  claude auth status --json \
  | python3 controlla_accesso.py --salva stato/con.json
```

❓ **Indovinate, a voce**: con una chiave impostata, anche finta, Claude Code userà ancora il vostro Pro?

🎞 **Nella registrazione del corso:** senza la chiave, collegato sì, metodo `claude.ai`: l'accesso con l'abbonamento; con la chiave finta, collegato sì, metodo `api_key`, chiave da `ANTHROPIC_API_KEY`. **La chiave**, non più il Pro

M4-D01 La chiave che scavalca il Pro

Demo del docente nel terminale, poi la vostra prova: nessun messaggio a Claude

> Dal riferimento delle variabili d'ambiente di Claude Code, su `ANTHROPIC_API_KEY`: «When set, this key is used instead of your Claude Pro, Max, Team, or Enterprise subscription even if you are logged in.»

La registrazione del corso, 28 ottobre 2026, Claude Code 2.1.312: la coppia, sull'account del docente, e il controllo dell'elemento sui due file salvati. È una **garanzia**: deve valere a ogni esecuzione. Qui, **confermata**

le righe del controllo: «[…]» segna un taglio

```
senza ANTHROPIC_API_KEY: collegato sì · metodo claude.ai
con ANTHROPIC_API_KEY (per quel solo comando): collegato sì ·
  metodo api_key · chiave da ANTHROPIC_API_KEY
Con ANTHROPIC_API_KEY impostata, Claude Code usa la chiave
  al posto dell'abbonamento: confermata
[…]
```

> Dalla pagina sull'autenticazione: «In non-interactive mode (-p), the key is always used when present.» Questa coppia non la mette alla prova: spiega perché gli script del corso tolgono la chiave prima di ogni `claude -p`

M4-D01 La chiave che scavalca il Pro

Demo del docente nel terminale, poi la vostra prova: nessun messaggio a Claude

1. Dalla cartella `esercitazioni`: `tools/prepara_esercizio.sh M4-D01`, poi `cd ~/corso-claude-code/M4-D01`. Controllate che nel terminale `ANTHROPIC_API_KEY` **non** ci sia, con il comando del passo 2 del README, che non stampa mai la chiave
2. Gli **stessi due comandi** della demo (passo 3); poi `python3 controlla_accesso.py`, che dice l'esito, e di nuovo il controllo del passo 2: la chiave finta è esistita **solo** per il suo comando
3. In Claude Code: `claude`, poi `/status` per vedere il metodo di accesso, poi `/exit` — **nessun messaggio** a Claude
4. Dalla cartella `esercitazioni`: `tools/verifica.sh M4-D01`

Chi non finisce in aula **continua da solo**, con il README: i passi sono gli stessi

Nel repository delle esercitazioni:
`m4_claude_code_agenti/s3_agente_nel_repository/m4_3_installare_configurare/M4-D01_chiave_che_scavalca/`

Le impostazioni: dove e chi vince

Cinque livelli, una precedenza; e le regole di un progetto

	livello	dove sta, e per chi vale
1	gestite	dall'organizzazione
2	riga di comando	il JSON passato con `--settings`, per quella sessione
3	progetto, locali	`.claude/settings.local.json`: voi soli, in quel progetto; Claude Code lo tiene **fuori da git** quando crea il file
4	progetto, condivise	`.claude/settings.json`: chiunque lavori nella cartella; in git, va nel **commit**
5	utente	`~/.claude/settings.json`

- La precedenza va dall'alto: un'impostazione di un livello più alto **vince** su quella di uno più basso; le liste come `permissions.allow`, invece, si **combinano** fra i file
- `~/.claude.json`, nella home, è **un altro file**: tiene lo stato dell'applicazione e le preferenze dell'interfaccia. Permessi, hook e variabili d'ambiente vanno in `~/.claude/settings.json`
- Le regole di un progetto: **deny** e **ask** valgono **subito**; le regole **allow**, solo dopo che avete dato **fiducia** alla cartella. In una sessione interattiva Claude Code ve lo chiede, quando lo avviate in una cartella che non avete ancora autorizzato

Il progetto pronto

Le regole del repository delle esercitazioni: i segreti fuori, i test consentiti

`.claude/settings.json`

```
{
  "permissions": {
    "deny": [
      "Read(**/.env)",
      "Read(**/.env.*)",
      "Read(**/secrets/**)",
      "Read(**/*.pem)",
      "Read(**/*.key)"
    ]
  }
}
```

- Il file del **repository delle esercitazioni**: lo riceve ogni spazio di lavoro che `tools/prepara_esercizio.sh` prepara, se l'elemento non porta un suo `.claude/`
- Cinque regole **deny** per lo strumento **Read**: i file `.env`, la cartella `secrets/`, i file `.pem` e `.key`. Le fa rispettare **Claude Code**, non il modello
- Gli esercizi che fanno girare i test portano in più una regola **allow** per il loro comando; in «Un file mai visto»: `Bash(tools/php.sh php progetti/gestionale-php/vendor/bin/phpunit *)`

- L'asterisco va **dopo il comando**: `Bash(git log *)` consente soltanto i comandi `git log`
- `/permissions`: vedere e cambiare le regole allow, ask e deny. E «Yes, and don't ask again» su un comando Bash o un dominio WebFetch salva la regola in `.claude/settings.local.json`, alla radice del repository git

Il ciclo delle esercitazioni

Prepara, lavora in Claude Code, verifica: per ogni demo e ogni esercitazione

```
# dalla cartella esercitazioni
git pull                              # gli elementi del modulo
tools/prepara_esercizio.sh <ID>       # lo spazio di lavoro
cd ~/corso-claude-code/<ID>
claude                                # il lavoro, con il README
cd -                                  # di nuovo in esercitazioni
tools/verifica.sh <ID>                # 0 fatto, 1 non ancora
tools/reset_esercizio.sh <ID>         # per ricominciare da capo
```

- Per ogni demo e ogni esercitazione: **prepara** uno spazio di lavoro nuovo, fuori dal repository, con un primo commit, così `git diff` mostra che cosa è cambiato; **lavora** in Claude Code con i passi del README; **verifica**; se serve, **ricomincia**
- La verifica è uno script dell'elemento, con **controlli deterministici**: file, test, uscite. 0 fatto, 1 non ancora, 2 manca qualcosa per partire
- Si lavora sui progetti **sintetici** del corso: il gestionale e la rete di **CiaspoLink**, l'operatore inventato del corso

Un accesso, nessuna chiave, un progetto con le sue regole: Claude Code lavora sul vostro Pro e solo dove glielo permettete.